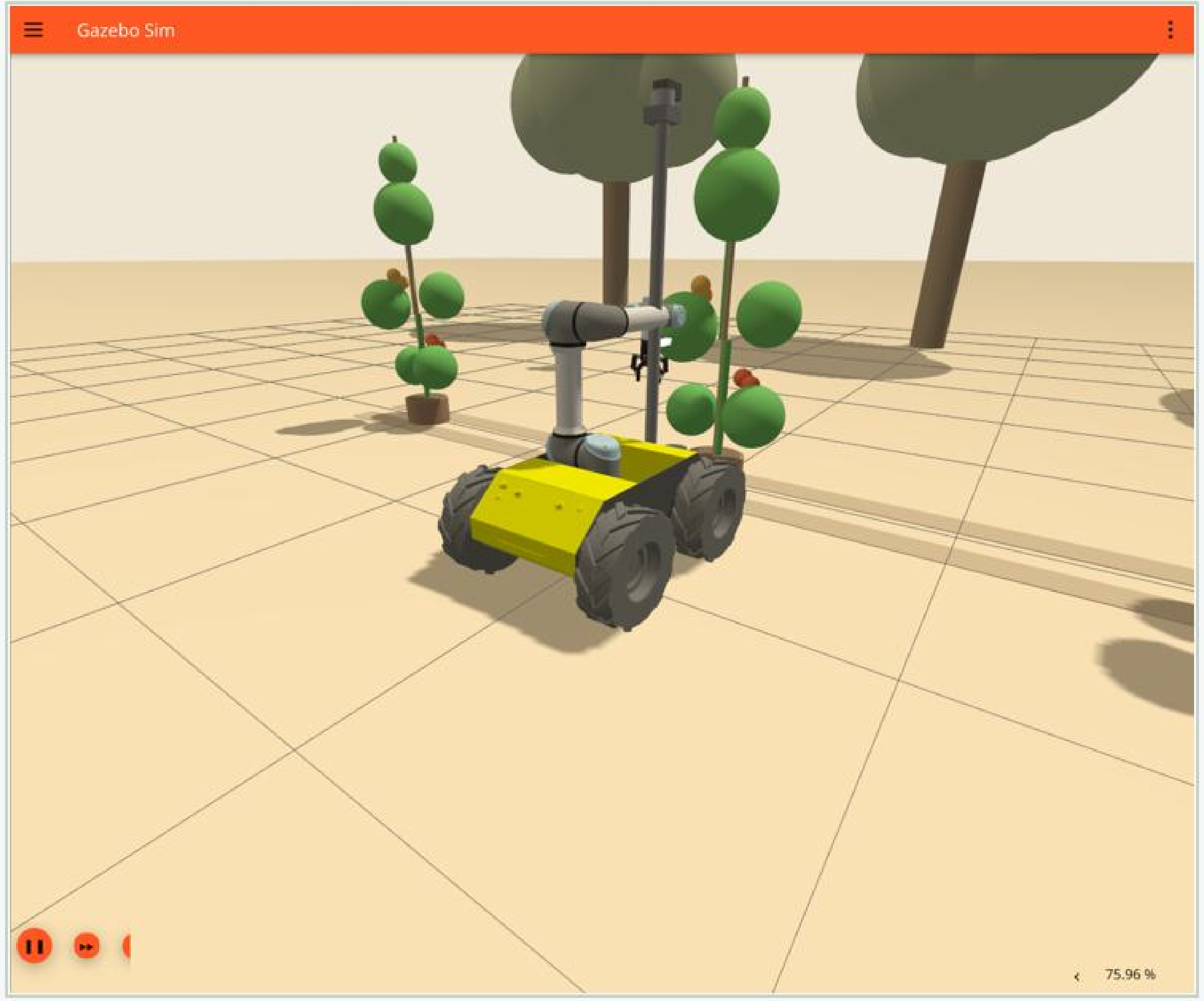
"Pick the red tomato" — a robot that obeys a sentence

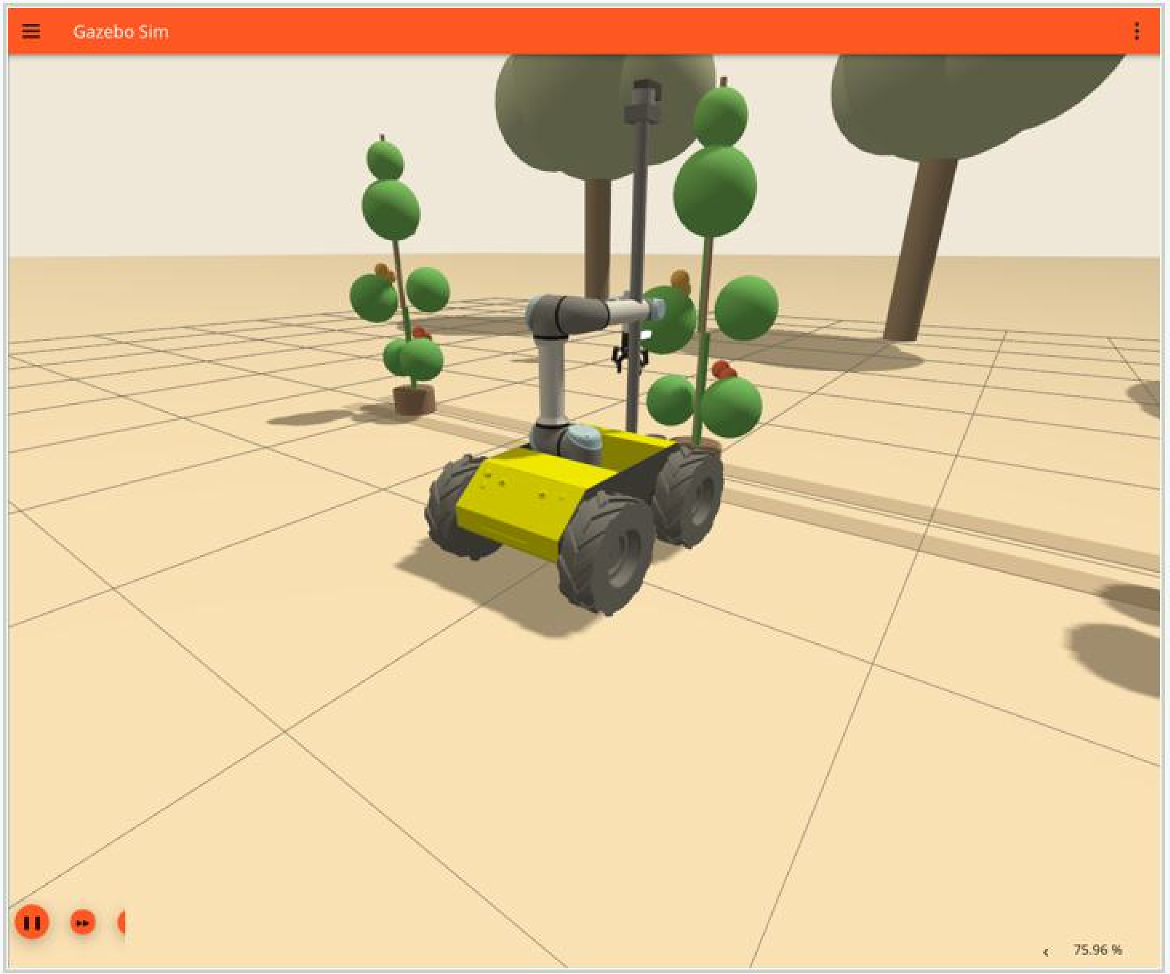
Day 2 demo, captured LIVE from the simulator · RAISE 2026 Summer School, ENET'Com Sfax



Everything on the next slides is a real capture: the Gazebo window, the terminals, the model outputs.

Step 1–2 · Start the world

Two terminals, one command each — then leave them alone



terminals 1 + 2

```
$ sim_d2      # terminal 1 — the greenhouse world (plant-row parking)
$ grasp_d3    # terminal 2 — the grasp service (it IS the grasp)
```

The robot parks facing a tomato plant — the exact spot the model was trained at (deterministic parking).

Step 3 · Install the brain

A fine-tuned SmolVLA checkpoint — one command, resumable, public

terminal 3

```
$  get_brain_d4
Downloading the reference brain (687 MB) — interruptions RESUME…
✓ reference brain installed — run:  vla_d4 --task C --spawn

# installs to ~/raise_checkpoints/smolvla_C_ref/checkpoints/006000/
#   pretrained_model/  =  model.safetensors (450M params) + processors
# skip it? first run auto-downloads from Hugging Face:
#   scalexi/smolvla-raise2026-ripeness-ref  (public, no key)
```

Trained in ~1.5 h on 49 auto-recorded demonstrations. All Day-2 tools auto-detect it.

Step 4 · Look inside ONE brain call

Three inputs, seven numbers out — that is the whole secret

terminal 3

```
$  vla_one_d4 --spawn

A VLA is a function:  action = model(image, instruction, state)

INPUT 1 — image  : 224×224 RGB from /wrist_camera/image_raw  (what it SEES)
INPUT 2 — words  : "pick the red tomato"                      (what we WANT)
INPUT 3 — state  : 7 floats from /joint_states                (where it IS)

OUTPUT  — action : 6 joint targets + gripper  (what to DO next)   [163 ms]

  joint           state (in)  action (out)   move?
  ------------------------------------------------
  shoulder_pan       +0.002       +0.058   ↑ up
  shoulder_lift      -1.560       -1.524   ↑ up
  elbow              +1.590       +1.537   ↓ down
  wrist_1            -1.570       -1.638   ↓ down
  gripper            +0.054       +0.003   ↓ down
```

The full executor is exactly this call, repeated 10 times per second.

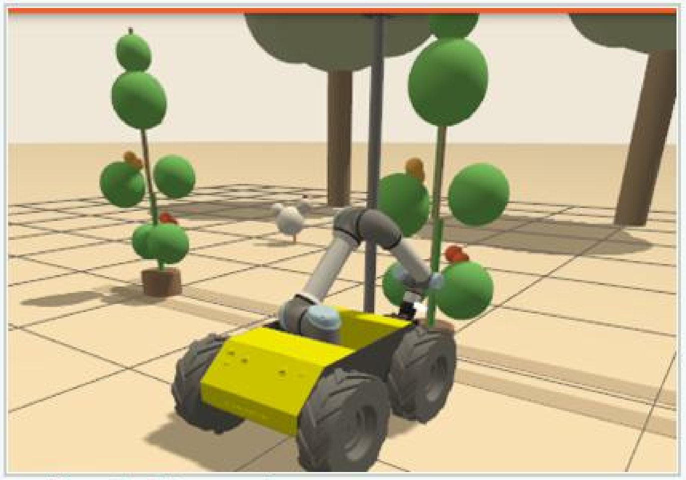
What the robot sees — and what it decides

The saved card (~/vla_one_step.png): camera frame beside the numbers

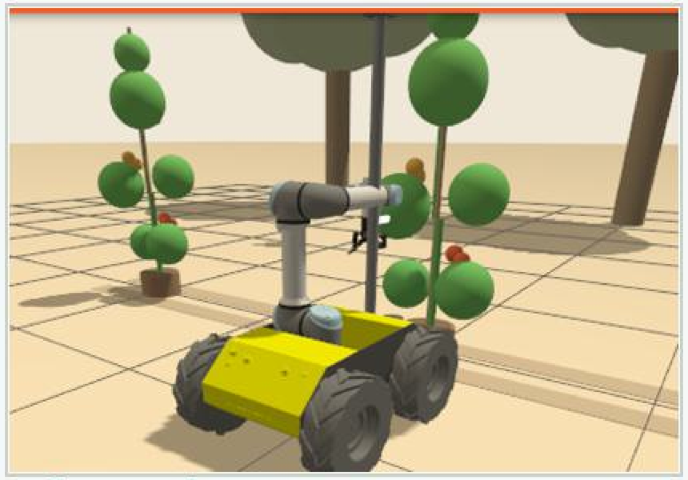
Left: the actual 224×224 wrist-camera frame the model received. Right: its inputs and output.

Step 5 · The pick — red tomato on the LEFT

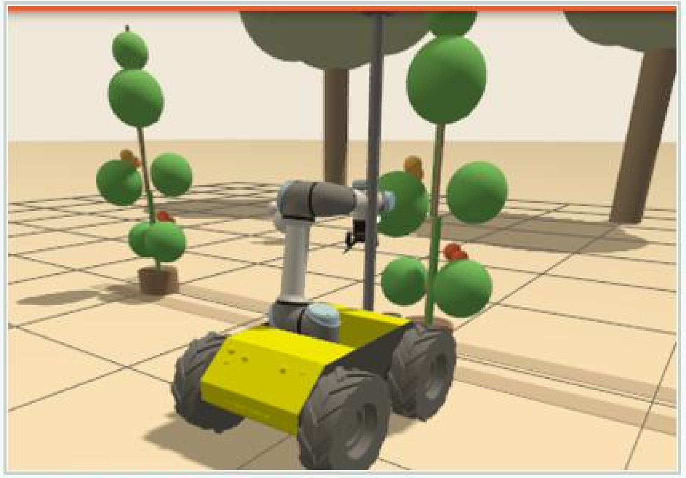
vla_d4 --task C --spawn --instruction "pick the red tomato"



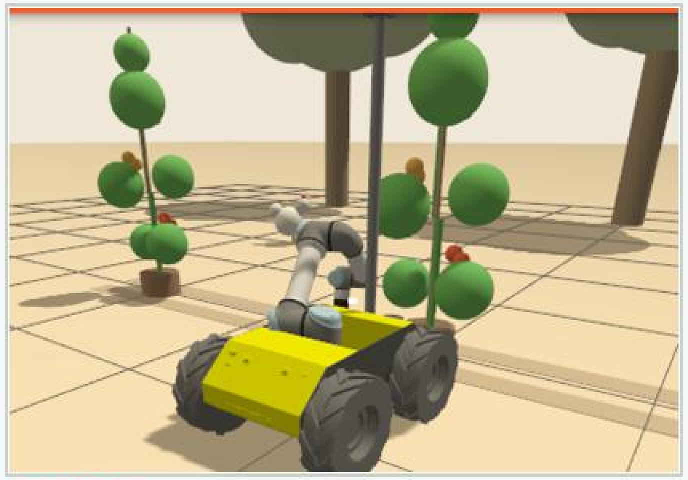
1 · scan above the left spot



2 · red seen → descend



3 · gripper closes on the red

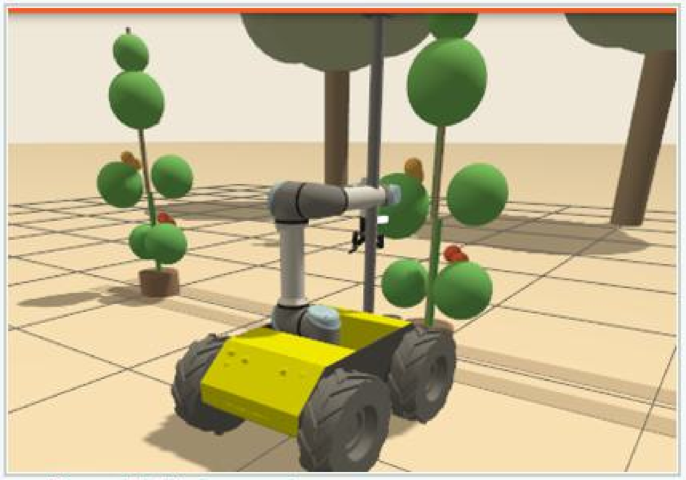


4 · hold — SUCCESS

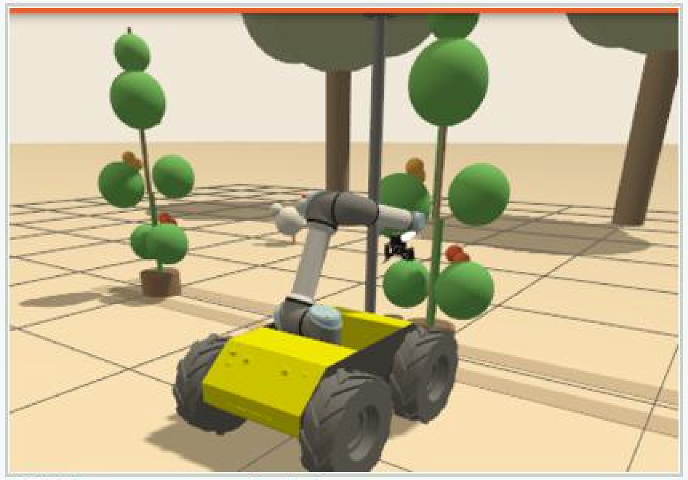
SUCCESS in 35 model calls — judged by the grasp service naming the RED tomato, not by looks.

Active perception — red tomato on the RIGHT

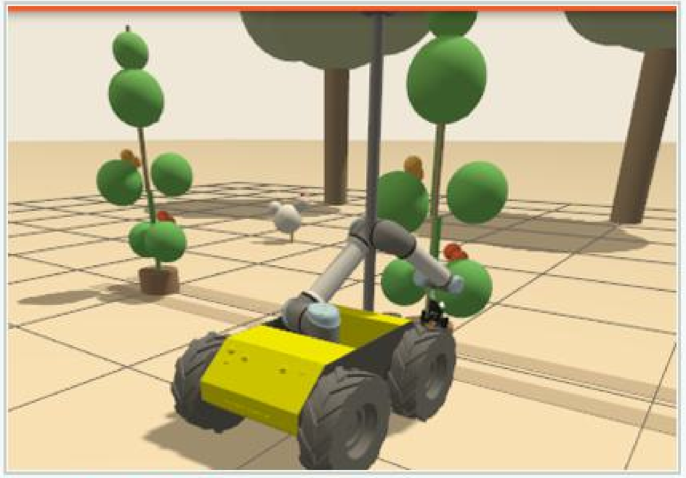
Same command, tomatoes swapped: watch it LOOK first



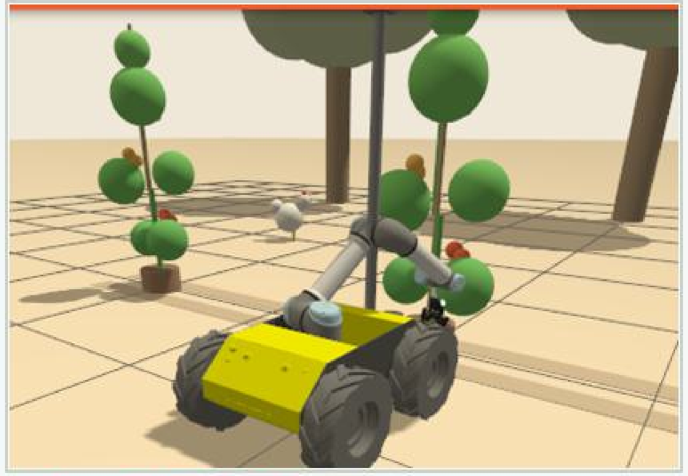
1 · scan above the LEFT spot…



2 · …sees GREEN → pans right



3 · descends on the right



4 · picks the red — SUCCESS

Nobody programmed the scan — it was learned from demonstrations that contained it.

Measured, not vibes: 100/100

The same rubric your own trained models are graded on

terminal 3 — the graded evaluation

```
$  ~/raise_venvs/lerobot/bin/python3 evaluator/evaluate.py --task C --trials 8

  trial  1/8  red=L  → success      (mean act   7.2 ms)
  trial  2/8  red=R  → success      (mean act   9.7 ms)
  ...
  trial  8/8  red=R  → success      (mean act   8.2 ms)

── score ─────────────────────────────────────
  success    : 8/8                     → 60/60
  latency    : max 167 ms (≤500)       → 20/20
  safety     : 8/8 no-wrong-grab       → 20/20
  TOTAL      : 100/100
```

8/8 correct picks alternating sides · 0 wrong grabs · max 167 ms per decision.

Run it yourself — everything is public

1. `git clone github.com/aniskoubaa/raise2026-student && ./start.sh`
   installs everything, opens the sim

2. `get_brain_d4`
   the trained model (or auto-downloads from Hugging Face on first run)

3. `vla_one_d4 --spawn`
   one brain call, explained

4. `vla_d4 --task C --spawn --instruction "pick the red tomato"`
   the pick you just watched

```
Open the step-by-step guide:   xdg-open lab_guides/lab2_handson.html
New to ROS? the gentler one:   xdg-open lab_guides/lab2_beginner_handson.html
```

Model card + weights: huggingface.co/scalexi/smolvla-raise2026-ripeness-ref